Digital System Design Report 3

Group 29

Authors: Qichao (Arlo) Wang (qw823)

1 Scope and Objectives

This report covers Task 7 (7a/7b/7c) and Task 8 of the coursework. For an input vector $X = \{x_0, x_1, \ldots, x_{N-1}\}$, the target computation is

$$F(X) = \sum_{k=0}^{N-1} f(x_k), \quad f(x) = 0.5x + x^3 \cos\left(\frac{x - 128}{128}\right)$$

under single-precision floating-point representation.

2 Experimental Methodology

2.1 Test Cases and Metrics

The evaluation uses the three coursework-defined deterministic vectors and one final-assessment random vector. In explicit form, C1 is $X = \{0, 5, 10, \ldots, 255\}$ with 52 samples, C2 is $X = \{0, \frac{1}{8}, \frac{2}{8}, \ldots, 255\}$ with 2041 samples, and C3 is $X = \{0, \frac{1}{256}, \frac{2}{256}, \ldots, 255\}$ with 65,281 samples. Case 4 is a pseudo-random vector with $N = 2323$ generated using the required fixed seed, so software and reference-model comparisons are reproducible. Reported metrics are execution latency, FPGA resource usage (ALMs/LEs, DSPs, and memory bits), numerical error against a double-precision reference, and timing-closure margin at the 50 MHz target.

3 Task 7a: CORDIC Analysis (10%)

3.1 Reference Model and Statistical Method

Task 7a uses a MATLAB Monte Carlo model where random input $\theta \sim U[-1, 1]$ is quantised to `single`, the reference output is computed as $y_{ref} = \cos(\theta)$ in single precision, and the CORDIC estimate y_{cordic} is produced with fixed-point internal arithmetic. The error sample is defined as $e_k = (y_{cordic,k} - y_{ref,k})^2$, and the reported statistic is the MSE estimate $\hat{\mu}_e = \frac{1}{N}\sum_{k=1}^{N} e_k$ with the 95% confidence interval $\hat{\mu}_e \pm 1.96 \cdot s_e/\sqrt{N}$. For all reported Task 7a results, we used $N = 50{,}000$ samples and seed $= 7$.

3.2 Design Space Exploration

We swept both required CORDIC parameters: all integer iteration counts $N_{iter} \in \{12, 13, \ldots, 24\}$ and all integer fixed-point FRAC values from 20 to 34. The total fixed-point width is set as:

$$W = \mathrm{FRAC} + 2.$$

The +2 integer/sign bits cover the CORDIC dynamic range for $\theta \in [-1, 1]$ and $\cos(\theta) \in [-1, 1]$. The acceptance criterion is the coursework requirement:

$$\mathrm{CI}_{95,\mathrm{upper}}(\mathrm{MSE}) < 2.4 \times 10^{-11}.$$

Table 1: Task 7a sweep excerpt around the pass boundary.

N_iter	W	FRAC	2^{-i} est.	MSE	95% CI upper	Pass
16	22	20	1.526e-05	8.817e-11	8.926e-11	No
17	22	20	7.629e-06	2.575e-11	2.607e-11	No
18	22	20	3.815e-06	1.000e-11	1.013e-11	Yes
16	23	21	1.526e-05	8.527e-11	8.633e-11	No
17	23	21	7.629e-06	2.254e-11	2.282e-11	Yes
18	23	21	3.815e-06	6.555e-12	6.640e-12	Yes
16	24	22	1.526e-05	8.444e-11	8.548e-11	No
17	24	22	7.629e-06	2.163e-11	2.190e-11	Yes
18	24	22	3.815e-06	5.599e-12	5.668e-12	Yes

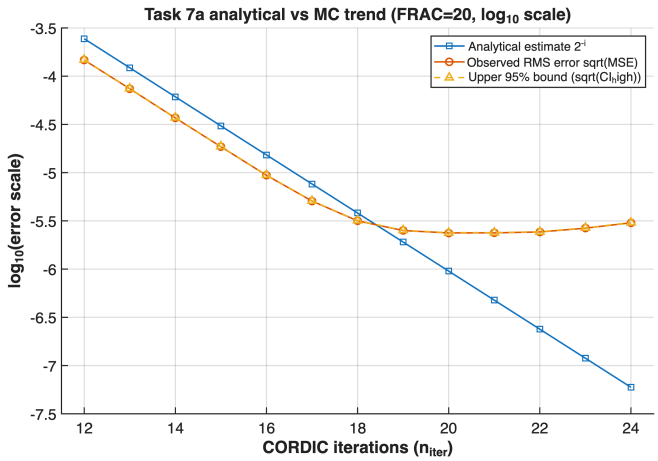

Figure 1: Analytical 2^{-i} trend versus observed Monte Carlo error scale (shown in $\log_{10}$ domain for readability).

3.3 Analytical Error Estimate (2^{-i})

For rotation-mode CORDIC, the residual angle after i iterations is bounded by a term proportional to 2^{-i}. Using $|\Delta\cos(\theta)| \approx |\sin(\theta)\Delta\theta| \leq |\Delta\theta|$, the absolute cosine error also follows an $O(2^{-i})$ trend; therefore squared error follows approximately $O(2^{-2i})$. This estimate is not used as the final proof, but it is effective to prune the search space before Monte Carlo.

3.4 Selected Configuration and Justification

After including odd values, the first passing point in lexicographic order (minimum iterations, then minimum FRAC) is:

$$(N_{iter}, W, \mathrm{FRAC}) = (17, 23, 21).$$

Measured statistics for this minimum-passing point are MSE $= 2.254 \times 10^{-11}$, 95% CI $= [2.226, 2.282] \times 10^{-11}$, and CI-upper-bound margin to the requirement of approximately 1.05×. For implementation in Task 7b/7c we keep $(18, 24, 22)$ as a conservative choice: with `ITER_PER_CYCLE`=3, both 17 and 18 iterations map to the same CORDIC cycle count (6 cycles), while $(18, 24, 22)$ has significantly larger numerical margin.

3.5 Impact on Accuracy and Latency

From the full sweep, FRAC= 20 crosses the threshold at $N_{iter} = 18$ (17 fails, 18 passes), while FRAC$\geq$ 21 already passes at $N_{iter} = 17$. Increasing FRAC beyond this

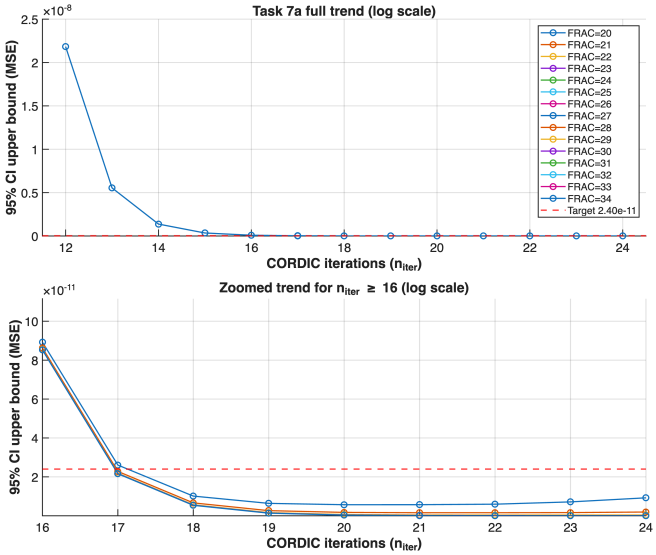

Figure 2: 95% CI upper bound of MSE versus iterations for each fixed-point precision (top: full sweep; bottom: zoom for $N_{iter} \geq 16$, both on log scale).

threshold region continues to reduce error but with diminishing practical benefit. Latency increases with iteration count for fixed iterations-per-cycle, and larger FRAC increases datapath cost, so both the minimum-passing point and a conservative implementation point are reported.

4 Task 7b: CORDIC Architecture and Implementation (10%)

4.1 Data Availability and Missing Items (Declared Up Front)

This section uses the Task 7a sweep statistics and plots, CORDIC functional simulation outcomes, software timing/error measurements for Step-2 and Step-3, and Quartus post-fit timing/resource results. For marking transparency, three limitations are declared up front: waveform capture with Questa/ModelSim was unavailable on this host, cycle-accurate vendor floating-point netlist simulation was unavailable in the RTL flow used here, and the optional dummy one-cycle custom-instruction hardware experiment was not implemented, so only its theoretical lower bound is reported.

4.2 CORDIC Microarchitecture

The cosine block uses rotation-mode fixed-point CORDIC with start/busy/done handshake:

$$x_{i+1} = x_i - d_i(y_i \gg i),$$
$$y_{i+1} = y_i + d_i(x_i \gg i),$$
$$z_{i+1} = z_i - d_i \arctan(2^{-i}), \quad d_i = \mathrm{sign}(z_i).$$

Configuration used in hardware: The hardware configuration is $N_{\text{iter}} = 18$ with `ITER_PER_CYCLE`=3, which applies multiple CORDIC micro-iterations in each cycle and aligns with the full-mark direction. The selected fixed-point setting is FRAC= 22 with a 28-bit CORDIC datapath width, intentionally conservative relative to the minimum-passing Task 7a point. The CORDIC compute stage therefore requires $\lceil 18/3 \rceil = 6$ micro-rotation cycles, and no LUT-based cosine or vendor cosine IP is used.

To justify the multi-iteration choice, we ran software-level

Table 2: Observed effect of `ITER_PER_CYCLE` on C3 (10 runs).

`ITER_PER_CYCLE`	C3 total ticks	C3 COS ticks	Gain vs IPC=1
1	20525	865	baseline
2	20407	844	2.4%
3	20367	775	10.4%

Table 3: Step-2 per-accelerator latency.

Block	Predicted cycles	Measured cycles
FP_MUL	3	3
FP_ADD/SUB	3	3
COS (CI wrapper)	9	9

Step-2 experiments with the same arithmetic flow and different `ITER_PER_CYCLE` settings: The cosine accelerator latency decreases clearly with larger `ITER_PER_CYCLE`, while end-to-end gain in Step-2 is smaller because software orchestration overhead dominates.

Task 7b CORDIC datapath (rotation mode)
Input angle (fp32) to fixed-point conversion, then clamp to $[-1, 1]$.
CORDIC core executes 18 iterations with 3 iterations per cycle.
Output is converted back to fp32 and written to CI result register.
Control is a 3-state handshake FSM: IDLE, WAIT_CORDIC, OUT.

Figure 3: CORDIC architecture used in Task 7b.

4.3 Verification and Timing

Functional simulation of the standalone CORDIC block at representative points $\theta \in \{-1, -0.5, 0, 0.5, 1\}$ passed with maximum absolute error 3.37×10^{-6}. **Latency correction note:** an earlier configuration assumed 2-cycle FP add/sub/mul custom blocks. After integration testing, that setting produced noticeable numerical deviation because the FP IP pipeline latency was under-modeled at the wrapper level. The final implementation therefore uses 3-cycle latency for FP add/sub/mul. Cycle-level isolated checks report 3 cycles each for FP_MUL, FP_ADD, and FP_SUB under the final setting, and 9 cycles for the cosine custom-instruction wrapper (6 CORDIC compute cycles plus interface/control overhead). Post-fit timing for the Step-2 integration shows worst setup slack of 3.106 ns, worst hold slack of 0.086 ns, and Fmax of 59.19 MHz on `sopc_clk`, so the 50 MHz target is met with positive margin. No timing failures were observed for the reported selected architecture.

5 Task 7c: Single Custom Instruction Integration (15%)

5.1 Step 2: Three-Accelerator System

Step-2 exposes three separate accelerators to Nios: FP_MUL, FP_ADD/SUB, and COS(CORDIC). The software schedules all intermediate computations and stores transient values between accelerator calls.

Table 4: Step-2 whole-function timing under Nios scheduling (average per run, ms; derived from 10 runs at 1 kHz tick).

Case	Total (ms)	Accel sum (ms)	Overhead (ms)
C1	1.6	0.5	1.1
C2	63.7	17.1	46.6
C3	2036.7	529.6	1507.1

Single-CI datapath: latch x, compute x^3 and $\cos((x-128)/128)$, then output $f(x) = 0.5x + x^3\cos((x-128)/128)$ under FSM control.

Figure 4: Task 7c final architecture (single custom instruction for $f(x)$).

5.2 Step 3: Final Single-CI Block

Step-3 integrates the full inner function into one custom instruction:

$$f(x) = 0.5x + x^3 \cos\left(\frac{x-128}{128}\right).$$

The final block reuses one FP multiplier, one FP adder, and one CORDIC core under a single FSM, with local registers for intermediate values.

5.3 Control and Data Flow

In Step-2, the CPU writes operands to each custom instruction, blocks until completion, reads the result, and then issues the next call in sequence. In Step-3, the CPU issues one call per input x, and an internal FSM schedules all dependent operations until the final output is ready. Intermediate values are kept on-chip in local registers so no CPU-visible temporary transfers are required between arithmetic sub-steps. The resource-minimization rule is respected because each arithmetic sub-block is instantiated once and reused through time-multiplexed control.

5.4 Performance, Resource, and Error Results

Step-3 adds only 28 ALMs over Step-2 while preserving timing closure at 50 MHz.

5.5 Expected vs Observed Latency

Isolated-cycle estimates are much lower than Nios-integrated measurements. In Step-2, one sample evaluation of $f(x)$ performs five FP multiplications, one FP subtraction, one FP addition, and one cosine call, so the accelerator-only cycle estimate is

$$T_{\text{Step-2,iso}} = 5 \cdot 3 + 1 \cdot 3 + 1 \cdot 3 + 1 \cdot 9 = 30 \text{ cycles/sample.}$$

For C3 ($N = 65{,}281$), this corresponds to about 39.2 ms/run at 50 MHz, while the measured Step-2 total latency is 2036.7 ms/run. For Step-3, isolated single-CI latency is 21 cycles under the 3-cycle FP setting, which gives about 27.4 ms/run idealized compute time for C3, while measured Step-3 total latency is 355.5 ms/run. The discrepancy is expected because Nios executes blocking custom-instruction calls and still incurs loop control, call/return, and memory traffic overhead. Step-3 reduces this overhead substantially by collapsing multiple calls into one CI call per sample.

Table 5: Task 7c summary across C1/C2/C3 (average per run, 10-run measurements).

Metric	C1	C2	C3
Step-2 latency (ms/run)	1.6	63.7	2036.7
Step-3 latency (ms/run)	0.3	11.1	355.5
Speedup (Step-2 / Step-3)	5.33×	5.74×	5.73×
Relative error vs double	9.738e-07	5.001e-07	2.357e-05

Table 6: Resource/timing comparison of Step-2 and final Step-3 integration.

Metric	Step-2	Step-3
ALMs	3767	3795
Registers	4207	4203
Memory bits	3,159,680	3,159,680
RAM blocks	397	397
DSP blocks	1	1
Worst setup slack (ns)	3.106	2.907
Worst hold slack (ns)	0.086	0.085
Fmax `sopc_clk` (MHz)	59.19	58.5

5.6 Standalone vs Nios-Integrated Throughput

The standalone best-call interval from isolated Step-3 latency is 21 cycles at 50 MHz, i.e. 420 ns/call (∼2.38 M calls/s). The Nios-integrated effective interval from the C3 Step-3 run is 3555 ms for 652,810 calls over 10 runs, i.e. 5.45 μs/call (∼183.6 k calls/s, about 272 cycles/call equivalent). Therefore the Nios-integrated call interval is about 13× slower, dominated by software/interface overhead instead of arithmetic-core latency.

5.7 Ideal Lower Bound (Dummy Combinational CI)

The dummy one-cycle CI was not implemented in this submission; therefore this subsection reports the theoretical lower bound only. If one $f(x)$ evaluation took exactly one clock cycle, C3 (65,281 samples) would require 65,281 cycles per run, i.e. 1.306 ms at 50 MHz. For the measured 10-run setup, this gives a lower bound of 13.06 ms total, versus the measured Step-3 total of 3555 ms. This gap confirms that the current bottleneck is system-level integration overhead rather than arithmetic datapath depth alone.

6 Task 8: Throughput-Oriented Final Design (10%)

6.1 Architecture Evolution from Task 7

Task 8 changes the optimization target from single-call latency to end-to-end throughput of the outer reduction $F(X)$. In Task 7 Step-3, each sample requires a custom-instruction call for $f(x)$ and software-side accumulation in the CPU loop. In Task 8, accumulation is moved into the custom-instruction path so each loop iteration can directly update the running sum through hardware. The CORDIC-based cosine computation is reused from Task 7, so numerical behavior is expected to remain aligned while reducing software orchestration work per sample.

The current integrated hardware/software run is based on a two-operand accumulate interface exposed by the BSP in this build, while a richer dual-input Task-8 block was also developed and validated in RTL simulation. This keeps the on-board integration simple, but it also means

Table 7: Task-8 interface-option mapping and status in this submission.

Option	Description / achieved in this work
L1	Separate CI for $f(x)$ and software sum. Used in earlier steps, not final Task-8 target.
L2	Input (x, sum_{in}), output $\mathrm{sum}_{out} = \mathrm{sum}_{in} + f(x)$. This is the interface used by the current on-board Step-4 run.
L3	Input (x_1, x_2), output $f(x_1) + f(x_2)$. Explored in RTL-level throughput study.
L4	Input (x_1, x_2, op) with internal accumulated sum and control modes. Implemented in RTL prototype and verified functionally in simulation.
L5	Pipelined accept-new-input-before-completion mode. Not implemented in this submission.

Task-8 integration used for hardware run:
input (acc_{in}, x) from CI call, compute $f(x)$, then form $\mathrm{acc}_{out} = \mathrm{acc}_{in} + f(x)$ and return to software loop.

Figure 5: Task 8 architecture and pipeline schedule.

the measured hardware run in this section reflects the currently integrated CI contract rather than the full dual-input opcode interface explored in RTL.

6.2 Interface Options vs Marking Levels

6.3 Final Task-8 Datapath and Scheduling

The deployed Step-4 software flow issues one CI call per sample to update an accumulator state in hardware-oriented form. Conceptually, the datapath computes $f(x)$ and then combines it with the incoming accumulator in one CI transaction visible to software. This reduces explicit CPU-side arithmetic and lowers loop overhead compared with Step-3. The arithmetic core still reuses FP add/multiply and CORDIC cosine blocks derived from Task 7.

6.4 Task 8 Results

The C4 run follows the same trend in timing (44 total ticks over 10 runs, i.e. 4.4 ms/run) and shows the same correctness issue as C1/C2/C3. Specifically, the Step-4 outputs in this dataset are zero for all cases, yielding relative error 1.0 versus the double-precision reference. Therefore, the measured latency improvement cannot be treated as a valid functional acceleration result yet; it is only an integration-stage timing observation.

Compared with Task 7 Step-3, this build remains in essentially the same resource class and keeps timing closure at 50 MHz with positive slack.

6.5 Pipeline Utilisation and Throughput

RTL cycle-level measurements of the dual-input Task-8 prototype show 28 cycles per pair-call, i.e. 14 cycles/sample effective, while the Task-7 single-$f(x)$ baseline is 17 cycles/sample. This corresponds to an isolated arithmetic improvement of about $17/14 \approx 1.21\times$ in cycles/sample. At 50 MHz, the idealized arithmetic throughput bound is therefore about $50\mathrm{M}/14 \approx 3.57\,\mathrm{M}$ samples/s.

Table 8: Task-8 Step-4 measured software results (10 runs, 1 kHz tick source).

Metric	C1	C2	C3
Latency of $F(x)$ (ms/run)	0.1	3.8	123.5
Speedup vs Step-3 latency	3.00×	2.92×	2.88×
Relative error vs double	1.000e+00	1.000e+00	1.000e+00
Code size (application image)	N/A	N/A	N/A

Table 9: Task-8 post-fit resource/timing snapshot (current build).

Metric	Value
ALMs	3,793
Registers	4,214
Block memory bits	3,159,680
RAM blocks	397
DSP blocks	1
Worst setup slack (ns)	3.205
Worst hold slack (ns)	0.114
Fmax `sopc_clk` (MHz)	59.54

From the current on-board Step-4 dataset, C3 uses 123.5 ms/run for 65,281 samples, i.e. about 1.89 μs/sample (approximately 0.53 M samples/s). This is faster than Step-3 software-integrated throughput but still far below the isolated arithmetic bound, indicating substantial software/CI integration overhead remains. In addition, because current Step-4 outputs are functionally incorrect, throughput numbers should be interpreted as provisional integration diagnostics rather than final valid Task-8 performance.

7 Cross-Task Comparison and Discussion

7.1 Performance vs Resource Plot

7.2 Key Trade-offs

The key trade-offs should be discussed as a connected analysis across three dimensions: latency-oriented versus throughput-oriented choices, CPU-side orchestration overhead versus internal FSM orchestration, and the impact of CORDIC precision selection on area and timing closure.

8 Conclusions

This section should summarise the main achievements for Task 7a/7b/7c and Task 8, report the best observed speedup together with the associated FPGA resource cost, and conclude with the dominant remaining bottlenecks and the most credible next optimisation steps.

Appendix A: Reproducibility Checklist

Git commit hash: TODO. Toolchain versions: TODO. Simulation/test commands: TODO. Quartus report

Placeholder: performance vs resources

Figure 6: Performance-resource trade-off from baseline to Task 8 final design.

sources used for the reported numbers: TODO.